K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Capstone Project

Smart City Traffic Management and Route Optimization System

Submitted By:

Tamanna

2401201138

BCA (AI & DS) - A

Submitted To:

Ms. Aarti Sangwan

1. Introduction

Smart city concepts. Smart cities use data and algorithms to manage transport, utilities and public services. A road network maps naturally onto a graph, so classic graph algorithms can answer routing and planning questions quickly and explainably.

Importance of traffic optimization. Congestion wastes time and fuel, delays emergency response and raises emissions. Fast shortest-path computation, rerouting under congestion and well-planned infrastructure reduce these costs.

2. Objectives

- Model a city as a weighted graph where intersections are vertices, roads are edges and the weight is distance or travel time.
- Find the shortest route between two intersections (Dijkstra) and show every step of the distance updates.
- Simulate congestion on chosen roads and check whether the best route changes (Bellman-Ford).
- Check connectivity and traversal order using BFS and DFS.
- Plan the cheapest road infrastructure using Minimum Spanning Tree algorithms (Prim and Kruskal).
- Compare the running time and memory of the algorithms on networks of 10, 25, 50 and 100 intersections.

3. System Design and Modules

The system is written in Python and presented as an interactive Streamlit web application with one tab for each module.

Module	File	Algorithms / Purpose
Road network modelling	graph_model.py	Adjacency list and adjacency matrix
Route planning	route_planner.py	Dijkstra with step-by-step log
Traffic analysis	traffic_analysis.py	Bellman-Ford, congestion simulation
Connectivity	connectivity_module.py	BFS, DFS
Infrastructure planning	mst_planner.py	Prim, Kruskal
Comparative analysis	benchmark.py	Time and memory on 10/25/50/100 nodes
User interface	app.py	Streamlit app, CSV import

4. Experimental Results

4.1 Road network

The sample city has 8 intersections (Airport, Central, Mall, Market, Station, Hospital, University and Stadium) joined by 12 roads. Users can add new intersections and roads or upload a CSV file in the format from,to,weight.

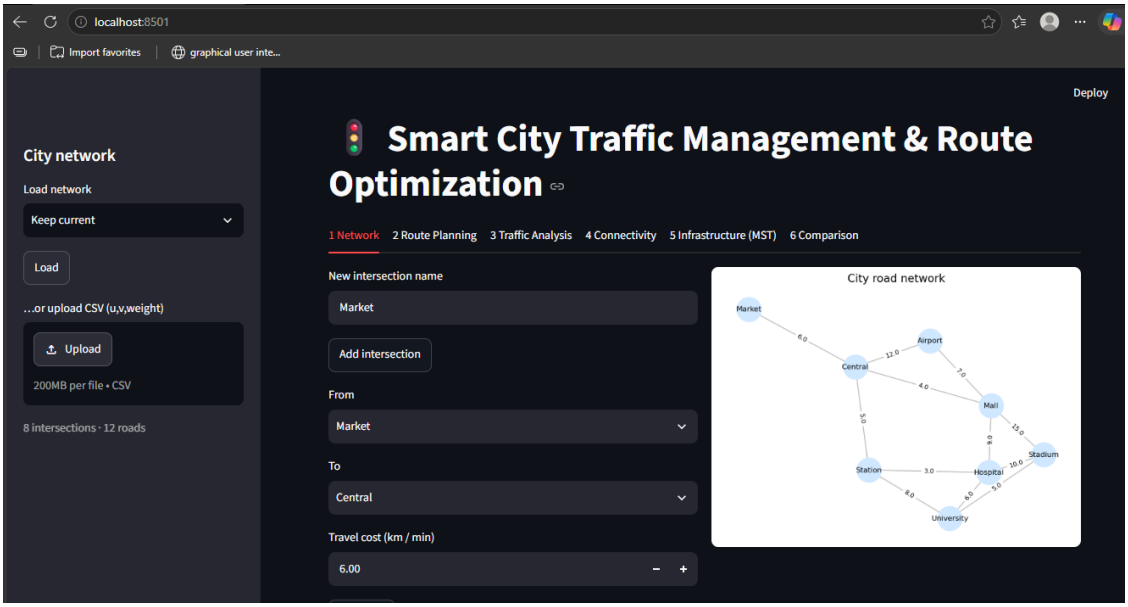


Figure 1: Network tab: add intersections and roads, city road network graph

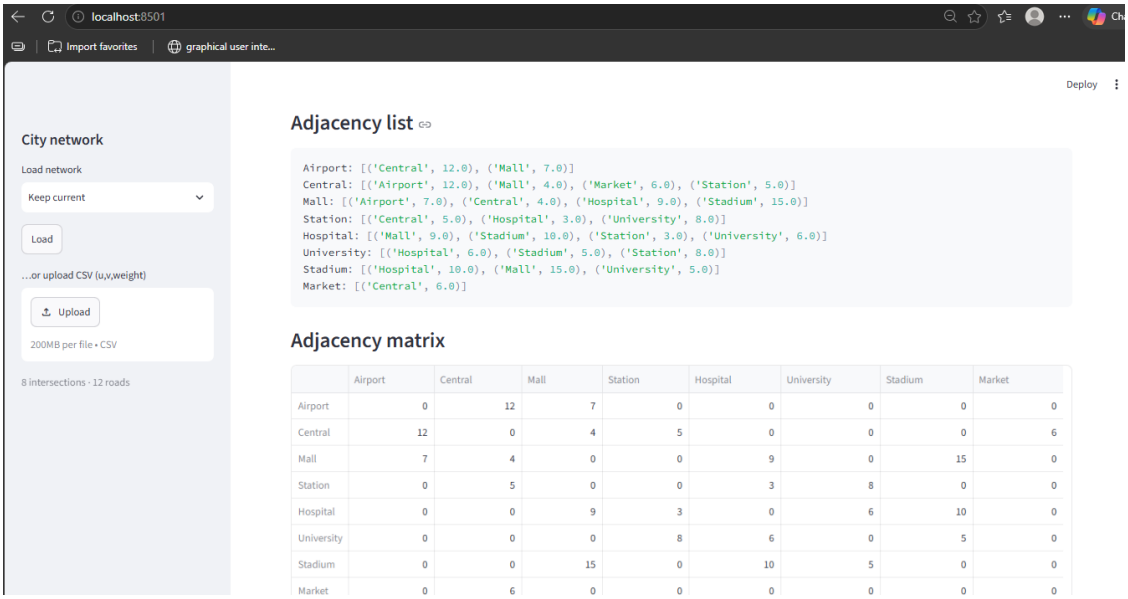


Figure 2: Adjacency list and adjacency matrix of the network

4.2 Route calculation (Dijkstra)

The shortest route from Airport to Stadium is **Airport → Mall → Stadium with total cost 22**. The step table shows how the tentative distances are updated at every visit.

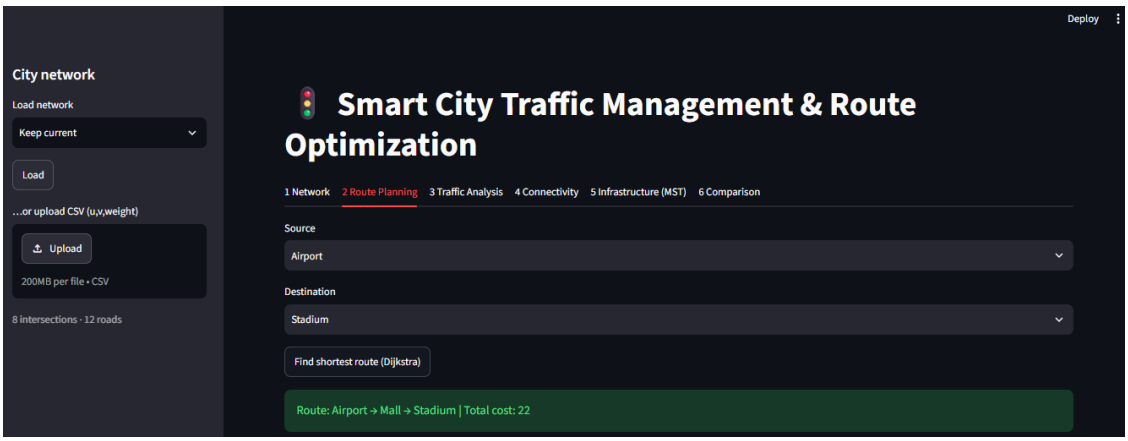


Figure 3: Route planning: shortest route found by Dijkstra

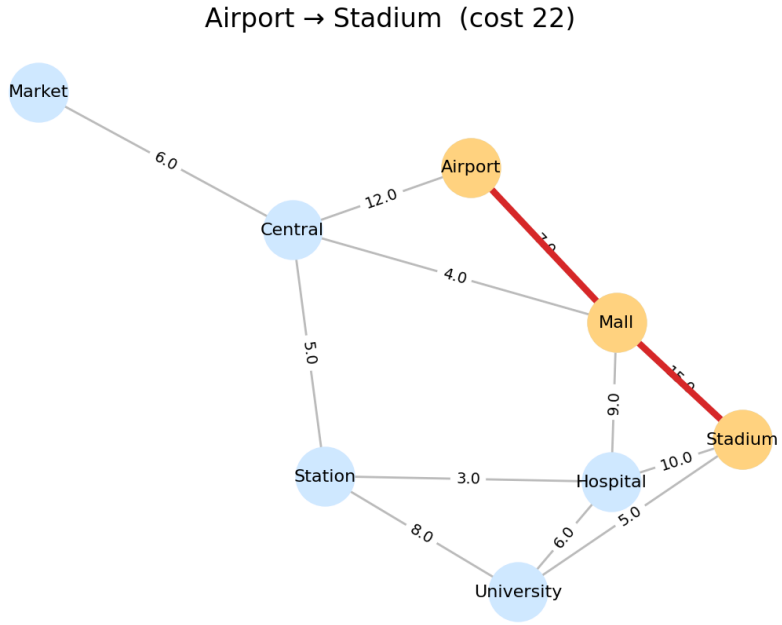


Figure 4: Shortest path highlighted on the city graph

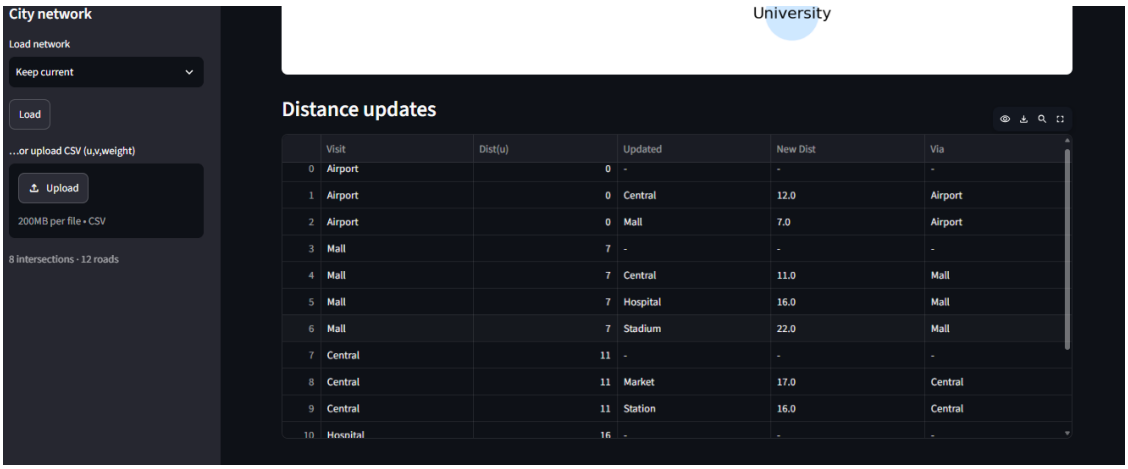


Figure 5: Dijkstra distance-update table

4.3 Traffic analysis (Bellman-Ford)

Congestion is applied to a chosen road (here Mall–Stadium with multiplier ×5). The original route cost was 22; after congestion the system **reroutes** through Airport → Mall → Hospital → Stadium with new cost 26, an increase of about 18%.

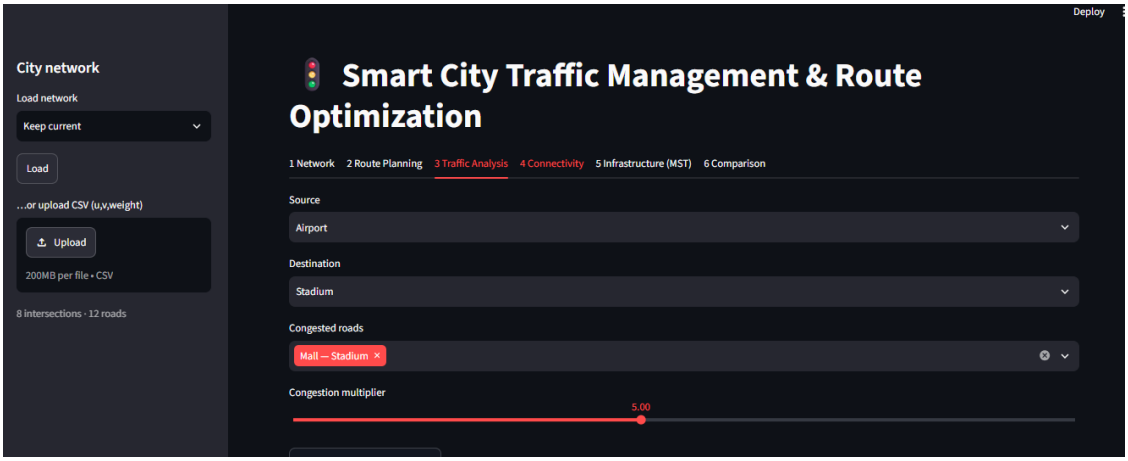


Figure 6: Traffic analysis: selecting congested road and multiplier

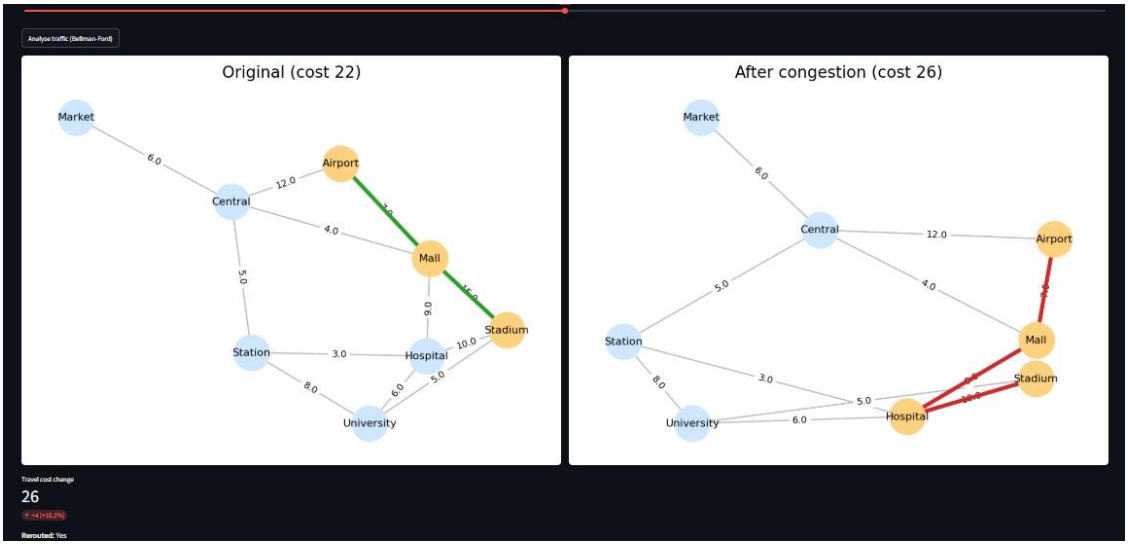
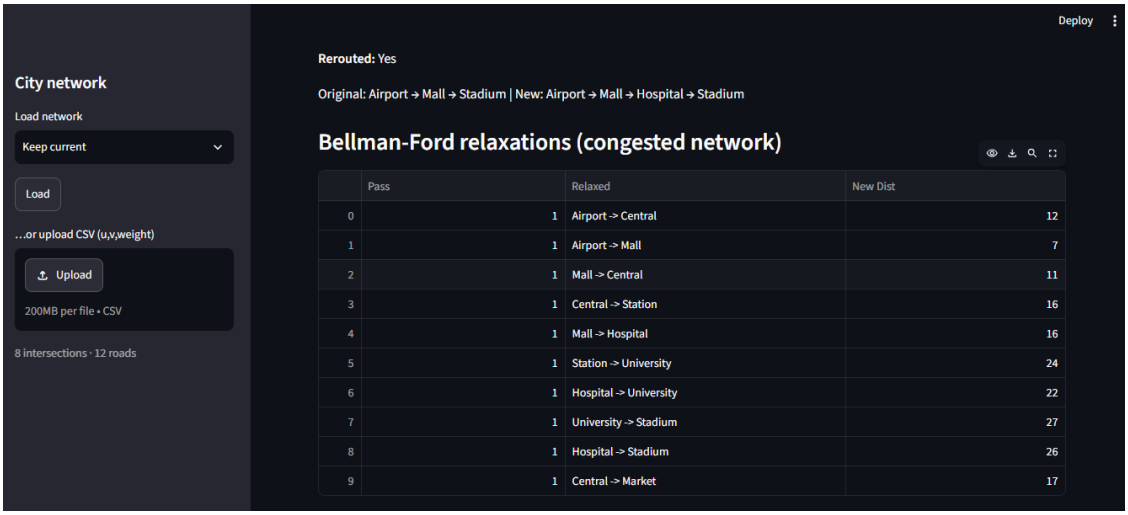


Figure 7: Original network (cost 22) versus congested network (cost 26)



	Pass	Relaxed	New Dist
0	1	Airport -> Central	12
1	1	Airport -> Mall	7
2	1	Mall -> Central	11
3	1	Central -> Station	16
4	1	Mall -> Hospital	16
5	1	Station -> University	24
6	1	Hospital -> University	22
7	1	University -> Stadium	27
8	1	Hospital -> Stadium	26
9	1	Central -> Market	17

Figure 8: Bellman-Ford relaxation table for the congested network

4.4 Connectivity (BFS and DFS)

- **BFS order:** Airport, Central, Mall, Market, Station, Hospital, Stadium, University
- **DFS order:** Airport, Central, Mall, Hospital, Stadium, University, Station, Market
- **Reachable:** all 8 intersections; no unreachable area, so the network is fully connected.

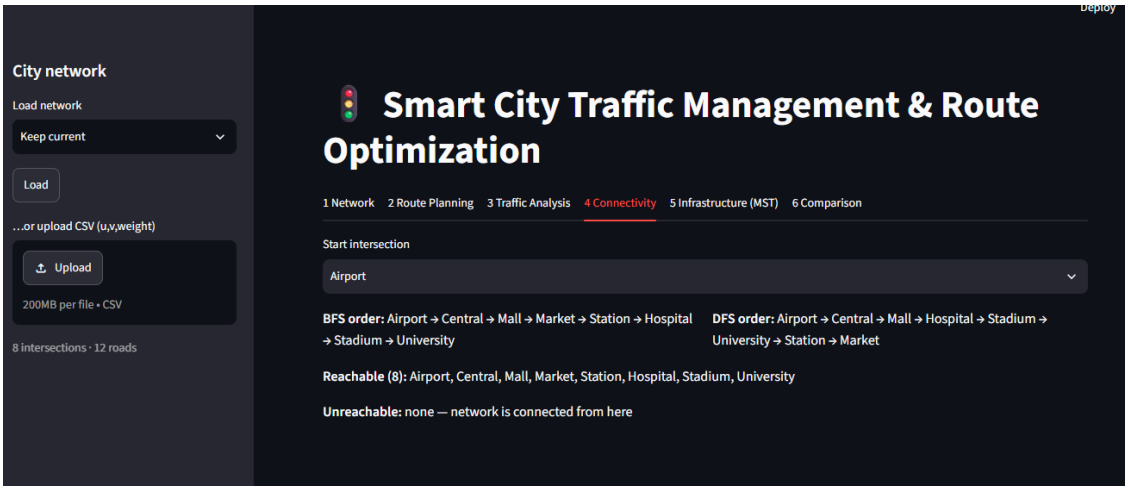


Figure 9: Connectivity tab: BFS and DFS traversal from Airport

4.5 Infrastructure planning (MST)

Prim and Kruskal both select the same minimum total infrastructure cost of **36**. The MST connects all junctions at the lowest cost but has no redundancy.

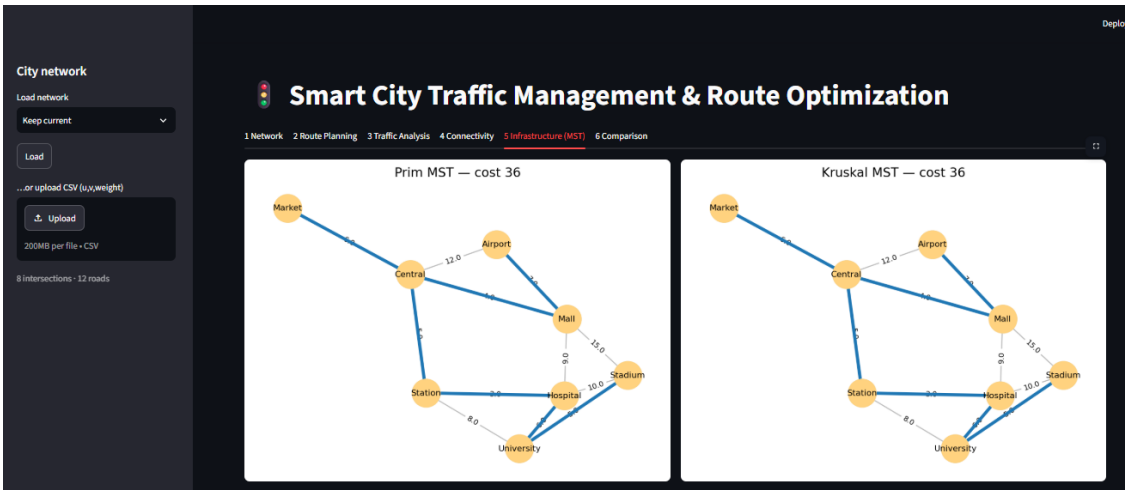


Figure 10: Prim MST and Kruskal MST (cost 36 each)

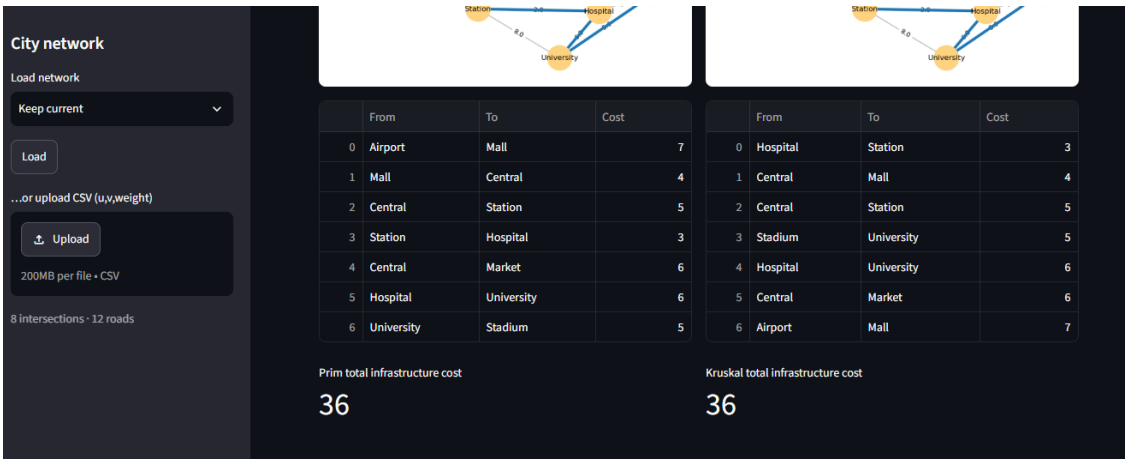


Figure 11: Roads selected by Prim and Kruskal

4.6 Performance comparison

The benchmark tab runs every algorithm on random networks of 10, 25, 50 and 100 intersections and records time and memory.

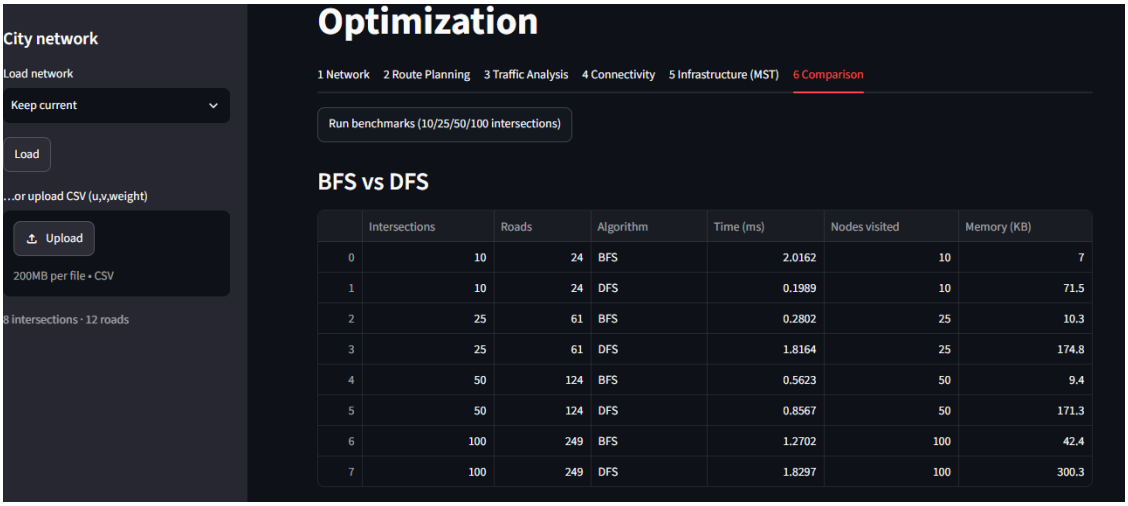


Figure 12: BFS versus DFS benchmark

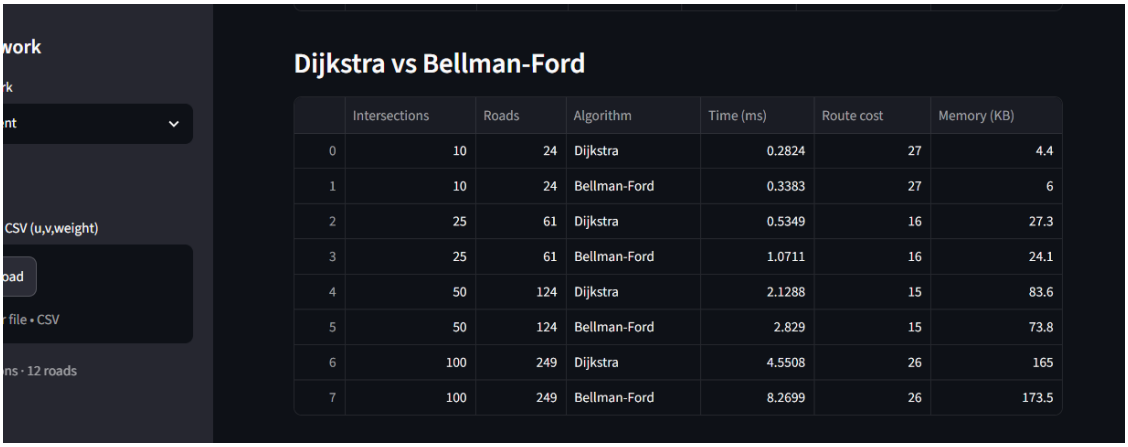


Figure 13: Dijkstra versus Bellman-Ford benchmark

Intersections	Roads	Dijkstra (ms)	Bellman-Ford (ms)
10	24	0.2824	0.3383
25	61	0.5349	1.0711
50	124	2.1288	2.8290
100	249	4.5508	8.2699



Figure 14: Prim versus Kruskal benchmark

Intersections	Roads	Prim (ms)	Kruskal (ms)
10	24	0.1408	0.0944
25	61	0.2371	0.1798
50	124	0.8748	0.3005
100	249	2.0680	0.7490

5. Analysis

- **BFS vs DFS:** both run in O(V+E). BFS uses a queue and finds the fewest-hop reachability, which suits “what is within N junctions” monitoring. DFS goes deep using recursion and suits exploring and connectivity checks.
- **Dijkstra vs Bellman-Ford:** Dijkstra, O((V+E) log V), was about twice as fast in the benchmark and is ideal for live navigation with non-negative weights. Bellman-Ford, O(VE), is slower but handles negative weights and detects negative cycles, so it suits what-if analysis.
- **Prim vs Kruskal:** both gave the same optimal cost on every network. Kruskal, O(E log E), was faster on these sparse graphs, while Prim is better for dense graphs.
- **Traffic observations:** increasing the weight of a main road either raises the cost of the same path or triggers an alternate route; the module reports the change in cost and percentage.

- **Infrastructure insight:** the MST is the cheapest way to connect all junctions, but with no redundancy it is a baseline rather than a final design.

6. Conclusion

Key findings: Dijkstra is best for routing, Bellman-Ford for flexible analysis, BFS/DFS for reachability, and Prim/Kruskal give equal-cost minimal networks.

Benefits: graph algorithms give fast, explainable and optimal decisions for navigation, rerouting and planning.

Future improvements: directed one-way roads, A* search, live traffic data, real map import (OpenStreetMap), time-dependent weights and MST with redundancy.